Task [linux + shell]

<u>Part 1: LVM (Logical Volume Manager):</u> LVM allows flexible disk management by abstracting physical storage into logical volumes that can be resized dynamically.

- Look for the second disk.

```
[root@system ~]# lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   75G  0 disk
├─sda1    8:1    0    1M  0 part
├─sda2    8:2    0  512M  0 part /boot
├─sda3    8:3    0   25G  0 part /
├─sda4    8:4    0    8G  0 part /home
└─sda5    8:5    0  3.9G  0 part [SWAP]
sdb       8:16   0   20G  0 disk
```

- Partition the second disk for LVM.

```
[root@system ~]# fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x222be9c3.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-41943039, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-41943039, default 4194303
9): +800M

Created a new partition 1 of type 'Linux' and of size 800 MiB.
```

```
Command (m for help): t
Selected partition 1
Hex code or alias (type L to list all): 8e
Changed type of partition 'Linux' to 'Linux LVM'.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

[root@system ~]# lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   75G  0 disk
├─sda1    8:1    0    1M  0 part
├─sda2    8:2    0  512M  0 part /boot
├─sda3    8:3    0   25G  0 part /
├─sda4    8:4    0    8G  0 part /home
└─sda5    8:5    0  3.9G  0 part [SWAP]
sdb       8:16   0   20G  0 disk
└─sdb1    8:17   0  800M  0 part
```

```
[root@system ~]# pvcreate /dev/sdb1
  Physical volume "/dev/sdb1" successfully created.
  Creating devices file /etc/lvm/devices/system.devices
[root@system ~]# pvs
  PV         VG Fmt  Attr PSize   PFree
  /dev/sdb1     lvm2 ---  800.00m 800.00m
```

- Create a volume group, and set 16M as extends

```
[root@system ~]# vgcreate -s 16M vgdata /dev/sdb1
  Volume group "vgdata" successfully created
[root@system ~]# vgdisplay vgdata
  --- Volume group ---
  VG Name               vgdata
  System ID
  Format                lvm2
  Metadata Areas        1
  Metadata Sequence No  1
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                0
  Open LV               0
  Max PV                0
  Cur PV                1
  Act PV                1
  VG Size               784.00 MiB
  PE Size               16.00 MiB
  Total PE              49
  Alloc PE / Size       0 / 0
  Free  PE / Size       49 / 784.00 MiB
  VG UUID               pQsuMS-xJ8c-9Uaj-akgO-E2Ub-kfK1-GUkzFE
```

.

due to slight rounding or metadata overhead, LVM reports `total PE = 49` and VG size = 784M.

- Divided a volume group containing 50 extends on volume group lv.

```
[root@system ~]# lvcreate -n lv -l 100%VG vgdata
  Logical volume "lv" created.
[root@system ~]# lvs
  LV   VG     Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv   vgdata -wi-a----- 784.00m
```

- Make it as ext4 file system.

mkfs creates a filesystem so the operating system can store and manage files.

```
[root@system ~]# mkfs.ext4 /dev/vgdata/lv
mke2fs 1.47.1 (20-May-2024)
Creating filesystem with 200704 4k blocks and 50176 inodes
Filesystem UUID: 5031cfb4-1129-4eb8-aa84-f3ea09e7e1af
Superblock backups stored on blocks:
        32768, 98304, 163840

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

[root@system ~]# blkid /dev/vgdata/lv
/dev/vgdata/lv: UUID="5031cfb4-1129-4eb8-aa84-f3ea09e7e1af" BLOCK_SIZE="4096" TYPE="ext4"
```

- Mounted automatically under /mnt/data.

Mount attaches a filesystem to the Linux directory tree so it becomes accessible.

`mount` alone → temporary, runtime only

`/etc/fstab` → persistent, survives reboots

```
[root@system ~]# mkdir /mnt/data
[root@system ~]# mount /dev/vgdata/lv /mnt/data
[root@system ~]# mount -a
[root@system ~]# mount | grep /mnt/data
/dev/mapper/vgdata-lv on /mnt/data type ext4 (rw,relatime,seclabel)
```

```
[root@system ~]# vim /etc/fstab
[root@system ~]# mount -a
```

```
#
# /etc/fstab
Tools ed by anaconda on Tue Oct 21 08:25:13 2025
#
# Accessible filesystems, by reference, are maintained under '/dev/disk/'.
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info.
#
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#
UUID=8cf60cf6-545c-484f-a118-fea556b8afad /                       xfs     defaults        0 0
UUID=79c151bc-54c1-4359-9eef-a16f934c4479 /boot                   xfs     defaults        0 0
UUID=eacae113-e599-44e8-ace2-8204d432b903 /home                   xfs     defaults        0 0
UUID=8a28f763-17b2-4f21-b337-61c6c833364e none                    swap    defaults        0 0
/dev/vgdata/lv                            /mnt/data               ext4    defaults        0 0
```

To get exactly 50 extents, each 16 MB, I increased the partition size to a little more than 800 MB, because LVM reserves some space for metadata. This ensures the volume group contains the required 50 usable extents.

```
[root@system ~]# fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): d
Selected partition 1
Partition 1 has been deleted.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-41943039, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-41943039, default 4194303
9): +2G

Created a new partition 1 of type 'Linux' and of size 2 GiB.

Command (m for help): w
```

```
[root@system ~]# vgcreate -s 16M vgdata /dev/sdb1
  Physical volume "/dev/sdb1" successfully created.
  Volume group "vgdata" successfully created
[root@system ~]# lvcreate -n lv -l 50 vgdata
WARNING: ext4 signature detected on /dev/vgdata/lv at offset 1080. Wipe it? [
y/n]: y
  Wiping ext4 signature on /dev/vgdata/lv.
  Logical volume "lv" created.
```

After fixing the partition size, I repeated the same steps used earlier: I initialized the physical volume, recreated the volume group with a 16 MB extent size, created the logical volume with exactly 50 extents, formatted it with ext4, mounted it on /mnt/data, and finally verified the changes using `lsblk`, `vgdisplay`, `lvdisplay`, and mount checks.

```
[root@system ~]# vgdisplay vgdata
  --- Volume group ---
  VG Name               vgdata
  System ID
  Format                lvm2
  Metadata Areas        1
  Metadata Sequence No  2
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                1
  Open LV               1
  Max PV                0
  Cur PV                1
  Act PV                1
  VG Size               1.98 GiB
  PE Size               16.00 MiB
  Total PE              127
  Alloc PE / Size       50 / 800.00 MiB
  Free  PE / Size       77 / 1.20 GiB
  VG UUID               bO1cnx-GeYk-1eQX-RXT2-G7ul-uYEI-df2fMs
```

Part 2: users, groups and permissions

1.

- Add user: user1.
- set uid=601.
- Password: redhat.
- The user's login shell should be non-interactive. (no ssh access to server)

```
[root@system ~]# useradd -u 601 -s /sbin/nologin user1
useradd warning: user1's uid 601 outside of the UID_MIN 5000 and UID_MAX 6000
0 range.
[root@system ~]# id user1
uid=601(user1) gid=6004(user1) groups=6004(user1)
```

```
[root@system ~]# useradd -u 601 -s /sbin/nologin user1 2>/dev/null
[root@system ~]# passwd user1
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
```

```
[root@system ~]# su - user1
This account is currently not available.
```

2.

- Add user1 to group TrainingGroup.

```
[root@system ~]# groupadd TrainingGroup
[root@system ~]# usermod -aG TrainingGroup user1
[root@system ~]# id user1
uid=601(user1) gid=6004(user1) groups=6004(user1),6005(TrainingGroup)
```

3.

- Add users: user2, user3.
- The Additional group of the two users: user2, user3 is the admin group Password: redhat.
- user 3 with root permissions.

```
[root@system ~]# groupadd admin
```

```
[root@system ~]# useradd -G admin user2
[root@system ~]# useradd -G admin user3
[root@system ~]# passwd user2
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@system ~]# passwd user3
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@system ~]# usermod -aG wheel user3
```

`wheel` is a special system group that allows users to run commands as root using `sudo`.

```
[root@system ~]# su - user3
Last login: Sun Nov 23 16:46:20 EET 2025 on pts/0
[user3@system ~]$ sudo whoami
[sudo] password for user3:
root
```

```
[root@system ~]# id user3
uid=6005(user3) gid=6008(user3) groups=6008(user3),10(wheel),6006(admin)
```

<u>Part 3: SSH</u> (Secure Shell) It is a secure way to connect to another Linux server remotely.

- Generate SSH key.
- connect to different VM without password.

```
[root@system ~]# su - user3
Last login: Sun Nov 23 19:29:39 EET 2025 on pts/1
[user3@system ~]$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user3/.ssh/id_rsa):
/home/user3/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/user3/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user3/.ssh/id_rsa
Your public key has been saved in /home/user3/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:2n+QMrx1a+CcVtAPZavnhrz3tBTgkMqEUdByaw/bVxQ user3@system.example.com
The key's randomart image is:
+---[RSA 4096]----+
|        o+.     E.|
|        .oo  . o. |
|        .o..+ +.. |
|         o+o = o. |
|        .So=o =.. |
|        o+.=o+.o .|
|       . .B B.= ..|
|         ..= * =..|
|          ..o.+ o.|
+----[SHA256]-----+
```

```
[user3@system ~]$ ssh-copy-id ldap@ipaserver.example.com
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/user3/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
(ldap@ipaserver.example.com) Password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'ldap@ipaserver.example.com'"
and check to make sure that only the key(s) you wanted were added.

[user3@system ~]$ ssh ldap@ipaserver.example.com
Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-dashboard
Last login: Thu Nov 27 09:17:24 2025 from 192.168.99.10
[ldap@ipaserver ~]$ exit
```

For this task, we used `user3` instead of root because logging in as root over SSH is insecure. `user3` is a normal user with sudo privileges, which allows performing necessary administrative tasks safely. We generated an SSH key with `ssh-keygen -t rsa -b 4096` (`-t rsa` specifies the RSA key type, `-b 4096` sets the key length to 4096 bits for strong security). The public key was copied to the remote IPA server (`ldap@ipaserver.example.com`) using `ssh-copy-id`, enabling passwordless SSH login.

Part 4: permissions

- Copy /etc/fstab to /var/tmp name admin. (File System Table) defines how and where filesystems are mounted automatically at boot

```
[root@system ~]# cp /etc/fstab /var/tmp/admin
[root@system ~]# chown user1:user1 /var/tmp/admin
[root@system ~]# chmod 700 /var/tmp/admin
[root@system ~]# ls -l /var/tmp/admin
-rwx------. 1 user1 user1 806 Nov 23 20:35 /var/tmp/admin
```

- user1 could read, write and modify it.

```
[user1@system ~]$ cd /var/tmp
[user1@system tmp]$ ls
admin
```

```
[user1@system tmp]$ ls -l admin
-rwx------. 1 user1 user1 806 Nov 23 20:35 admin
[user1@system tmp]$ echo "test" >> admin
[user1@system tmp]$ cat admin

#
# /etc/fstab
# Created by anaconda on Tue Oct 21 08:25:13 2025
#
# Accessible filesystems, by reference, are maintained under '/dev/disk/'.
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info.
#
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#
UUID=8cf60cf6-545c-484f-a118-fea556b8afad /                       xfs     def
aults        0 0
UUID=79c151bc-54c1-4359-9eef-a16f934c4479 /boot                   xfs     def
aults        0 0
UUID=eacae113-e599-44e8-ace2-8204d432b903 /home                   xfs     def
aults        0 0
UUID=8a28f763-17b2-4f21-b337-61c6c833364e none                    swap    def
aults        0 0
/dev/vgdata/lv                            /mnt/data               ext4    def
aults        0 0
test
```

- user2 can't do any permission.

```
[user2@system ~]$ cd /var/tmp
[user2@system tmp]$ cat admin
cat: admin: Permission denied
[user2@system tmp]$ echo "test" >> admin
-bash: admin: Permission denied
```

Part 5: permissions

Security-Enhanced Linux: SELinux is a mandatory access control system that adds an extra layer of security to Linux. Enforcing blocks access, Permissive logs only, Disabled turns SELinux off.

- SELinux must be running in the Enforcing mode (permanent even after reboot).

```
[root@system ~]# vim /etc/selinux/config
```

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#     enforcing - SELinux security policy is enforced.
#     permissive - SELinux prints warnings instead of enforcing.
#     disabled - No SELinux policy is loaded.
# See also:
# https://docs.fedoraproject.org/en-US/quick-docs/getting-started-with-selinux/#getting-star
#
# NOTE: In earlier Fedora kernel builds, SELINUX=disabled would also
# fully disable SELinux during boot. If you need a system with SELinux
# fully disabled instead of SELinux running with no policy loaded, you
# need to pass selinux=0 to the kernel command line. You can use grubby
# to persistently set the bootloader to boot with selinux=0:
#
#    grubby --update-kernel ALL --args selinux=0
#
# To revert back to SELinux enabled:
#
#    grubby --update-kernel ALL --remove-args selinux
#
SELINUX=enforcing
# SELINUXTYPE= can take one of these three values:
#     targeted - Targeted processes are protected,
#     minimum - Modification of targeted policy. Only selected processes are protected.
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted


~
~
~
~
~
~
:wq
```

Then systemctl reboot

```
[root@system ~]# getenforce
Enforcing
[root@system ~]# sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
[root@system ~]#
```

If use setenforce 1 # 1 = enforcing, 0 = permissive → This changes SELinux mode immediately, but won't persist after reboot.

Part 6: bash script and processes

- Write a shell script that will keep running for 10 mins in the background.
- check the process that it's created and try to kill using commands.

```
[root@system ~]# vim monitor_process.sh
[root@system ~]# chmod +x monitor_process.sh
[root@system ~]# ./monitor_process.sh
Started background process with PID: 235646
Process 235646 is still running. Attempting to kill it...
Process 235646 killed.
./monitor_process.sh: line 21: 235646 Killed                  sleep 600
Process 235646 has already exited.
Process 235646 has already exited.
10 minutes are over. Script finished.
```

ps → check if process exists

```
#!/bin/bash
# monitor_process.sh

# Start a long-running process in the background (example: sleep 600)
sleep 600 &
PID=$!
echo "Started background process with PID: $PID"

# Run the script for 10 minutes
START_TIME=$(date +%s)
END_TIME=$((START_TIME + 600))

while [ $(date +%s) -lt $END_TIME ]; do
    if ps -p $PID > /dev/null; then
        echo "Process $PID is still running. Attempting to kill it..."
        kill -9 $PID && echo "Process $PID killed."
    else
        echo "Process $PID has already exited."
    fi
    sleep 10  # prevent CPU spike
done

echo "10 minutes are over. Script finished."
```

Part 7: Yum Repo

- Install tmux on your machine.

`tmux` is a terminal multiplexer: run multiple terminal sessions in one window. Can detach and reattach sessions. Useful for long-running tasks like installing packages or building a repo

```
[root@system ~]# yum install -y tmux
Last metadata expiration check: 0:42:44 ago on Thu 27 Nov 2025 12:49:34 PM EE
T.
Package tmux-3.3a-13.20230918gitb202a2f.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

- Install apache server on your machine(httpd) and Install mysql.
  httpd (Apache) → will host local repo (Zabbix RPMs) via HTTP
  MariaDB → database server (needed for Zabbix later)

```
[root@system ~]# yum install -y httpd
Last metadata expiration check: 0:26:26 ago on Thu 27 Nov 2025 09:53:24 PM EE
T.
Package httpd-2.4.63-1.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@system ~]# systemctl enable --now httpd
[root@system ~]# yum install -y mariadb mariadb-server
Last metadata expiration check: 0:26:40 ago on Thu 27 Nov 2025 09:53:24 PM EE
T.
Package mariadb-3:10.11.11-1.el10.x86_64 is already installed.
Package mariadb-server-3:10.11.11-1.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@system ~]# systemctl enable --now mariadb
```

- Create a local yum repository on your local machine(available publicly) with the zabbix rpms: https://repo.zabbix.com/zabbix/4.4/rhel/7/x86_64/.

```
[root@system ~]# mkdir -p /var/www/html/zabbix
[root@system ~]# cd /var/www/html/zabbix
```

  curl → command-line tool to **download or transfer data** from a URL
  `-L` → **follow redirects** if the URL points to another location
  `-O` → **save the file with the same name** as in the URL

```
[root@system zabbix]# curl -LO https://repo.zabbix.com/zabbix/4.4/rhel/7/x86_
64/zabbix-server-mysql-4.4.0-1.el7.x86_64.rpm
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
  1 2490k    1 32768    0     0  23467      0  0:01:48  0:00:01  0:01:47 2347
 48 2490k   48 1199k    0     0   452k      0  0:00:05  0:00:02  0:00:03  452
 75 2490k   75 1871k    0     0   550k      0  0:00:04  0:00:03  0:00:01  550
100 2490k  100 2490k    0     0   653k      0  0:00:03  0:00:03 --:--:--  653
k
[root@system zabbix]# curl -LO https://repo.zabbix.com/zabbix/4.4/rhel/7/x86_
64/zabbix-agent-4.4.0-1.el7.x86_64.rpm
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
100  418k  100  418k    0     0   273k      0  0:00:01  0:00:01 --:--:--  273
k
[root@system zabbix]# curl -LO https://repo.zabbix.com/zabbix/4.4/rhel/7/x86_
64/zabbix-web-4.4.0-1.el7.noarch.rpm
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
 40 2977k   40 1199k    0     0   640k      0  0:00:04  0:00:01  0:00:03  640
100 2977k  100 2977k    0     0  1059k      0  0:00:02  0:00:02 --:--:-- 1059
k
[root@system zabbix]# curl -LO https://repo.zabbix.com/zabbix/4.4/rhel/7/x86_
64/zabbix-web-mysql-4.4.0-1.el7.noarch.rpm
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
nt
```

```
[root@system zabbix]# createrepo /var/www/html/zabbix
Directory walk started
Directory walk done - 4 packages
Temporary output repo path: /var/www/html/zabbix/.repodata/
Pool started (with 5 workers)
Pool finished
```

```
[root@system zabbix]# firewall-cmd --add-service=http --permanent
Warning: ALREADY_ENABLED: http
success
[root@system zabbix]# firewall-cmd --reload
success
[root@system zabbix]# systemctl restart httpd
```

```
[root@system zabbix]# curl -I http://localhost/zabbix/
HTTP/1.1 200 OK
Date: Thu, 27 Nov 2025 20:44:12 GMT
Server: Apache/2.4.63 (Red Hat Enterprise Linux)
Content-Type: text/html;charset=ISO-8859-1
```

```
[root@system zabbix]# vim /etc/yum.repos.d/local-zabbix.repo
[root@system zabbix]# cat /etc/yum.repos.d/local-zabbix.repo
[local-zabbix]
name=Local Zabbix Repository
baseurl=http://localhost/zabbix
enabled=1
gpgcheck=0

[root@system zabbix]# yum repolist
repo id                                   repo name
AppStream                                 AppStream-Repo
BaseOs                                    BaseOS-Repo
local-zabbix                              Local Zabbix Repository
```

- Disable all other repositories and keep only the new repo.
  Every repository has a `.repo` file in `/etc/dnf/repos.d/`.I can disable them by editing the files or using commands:

```
[root@system zabbix]# dnf config-manager --disable AppStream
```

```
[root@system zabbix]# dnf config-manager --disable BaseOs
```

```
[root@system zabbix]# dnf repolist enabled
repo id                                   repo name
local-zabbix                              Local Zabbix Repository
[root@system zabbix]#
```

- Install zabbix rpms from the new repo (Download zabbix, zabbix-web,php, zabbix-server,zabbix-agent rpm's and their dependencies).

```
[root@system zabbix]# dnf install zabbix-server-mysql zabbix-web-mysql zabbix
-agent
Last metadata expiration check: 0:00:47 ago on Thu 27 Nov 2025 11:44:38 PM EE
T.
```

`zabbix-agent` can be installed directly from the Zabbix repo because it has no external dependencies.
`zabbix-server` and `zabbix-web` require additional system packages (PHP modules, OpenIPMI, unixODBC, etc.) that are not included in the Zabbix repo, so you must ensure EPEL or base system repos are also enabled to satisfy these dependencies.

Part 8: Network management

- Open port 443 , 80.

  Port 80: HTTP (HyperText Transfer Protocol). Used for normal, unencrypted web traffic. Port 443: HTTPS (HTTP Secure). Used for secure, encrypted web traffic. Opening them allows users to access websites hosted on the server.

  A firewall is a security system that controls and filters network traffic to protect servers and networks.

- Make the changes permanent.

```
[root@system ~]# firewall-cmd --permanent --add-service=http
success
```

```
[root@system ~]# firewall-cmd --permanent --add-service=https
success
[root@system ~]# firewall-cmd --reload
success
```

```
[root@system ~]# firewall-cmd --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: enp0s3 enp0s8
  sources:
  services: cockpit dhcpv6-client http https ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

- Block ssh connection for your colleague ip to the VM.

To prevent unauthorized or restricted users from accessing the server via SSH

```
[root@system ~]# firewall-cmd --permanent --add-rich-rule='rule family="ipv4"
 source address="192.168.1.100" port protocol="tcp" port="22" reject'
success
[root@system ~]# firewall-cmd --reload
success
[root@system ~]# firewall-cmd --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: enp0s3 enp0s8
  sources:
  services: cockpit dhcpv6-client http https ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
        rule family="ipv4" source address="192.168.1.100" port port="22" prot
ocol="tcp" reject
```

Part 9: Cronjob

- Create a cronjob that will run at 1:30 AM every day.
  Cron is a Linux scheduler that automatically runs commands or scripts at specified times or intervals
- collect the users logged in and save them in a file. Format : timestamp – users.

Note: the cronjob can run a script.

```
[root@system ~]# vim /usr/local/bin/log_users.sh
[root@system ~]# chmod 0755 /usr/local/bin/log_users.sh
[root@system ~]# vim /var/log/logged_users.log
[root@system ~]# chmod 0644 /var/log/logged_users.log
[root@system ~]# /usr/local/bin/log_users.sh
```

```
[root@system ~]# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[root@system ~]# crontab -l
30 1 * * * /usr/local/bin/log_users.sh
```

`tr` cleans up spaces, `cut` extracts the first column (username), and `sed` removes trailing spaces, producing a clean list of unique logged-in users.

```
#!/bin/bash
# /usr/local/bin/log_users.sh
LOGFILE="/var/log/logged_users.log"
DATE=/usr/bin/date
WHO=/usr/bin/who
TR=/usr/bin/tr
CUT=/usr/bin/cut
SORT=/usr/bin/sort
UNIQ=/usr/bin/uniq
SED=/usr/bin/sed

TIMESTAMP=$($DATE "+%Y-%m-%d %H:%M:%S")
# get first column from who (collapse spaces, cut field1), sort & uniq, join on one line
USERS=$($WHO | $TR -s ' ' | $CUT -d' ' -f1 | $SORT | $UNIQ | $TR '\n' ' ')
```

```
echo "$TIMESTAMP - $USERS" >> "$LOGFILE"
```

```
[root@system ~]# cat /var/log/logged_users.log
2025-11-27 16:56:53 - root yasmin
2025-11-27 16:58:02 - root yasmin
2025-11-28 01:30:02 - yasmin
```

Part 10: Mariadb

- install mariadb from the local repo that was created earlier.

```
[root@system zabbix]# dnf config-manager --enable local-zabbix
[root@system zabbix]# dnf install mariadb-server mariadb -y
Local Zabbix Repository                         344 kB/s | 1.5 kB     00:00
Package mariadb-server-3:10.11.11-1.el10.x86_64 is already installed.
Package mariadb-3:10.11.11-1.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

- open ports in iptables from mariadb.
  Allow inbound connections to port 3306/tcp in the firewall on the server where MariaDB is running.
  This can be done using either `firewall` or `iptables`.
  To allow remote or local clients to connect to MariaDB/MySQL database.

```
[root@system zabbix]# systemctl enable --now mariadb
[root@system zabbix]# firewall-cmd --permanent --add-service=mysql
success
[root@system zabbix]# firewall-cmd --reload
success
[root@system zabbix]# firewall-cmd --list-all | grep mysql
  services: cockpit dhcpv6-client http https mysql ssh
```

- create database , user(note: handle permissions).

```
[root@system zabbix]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 28
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement
.

MariaDB [(none)]> CREATE DATABASE studentdb;
Query OK, 1 row affected (0.003 sec)
```

To ensure the user can perform only allowed operations on the database.

% allows the user to connect from any host.

```
MariaDB [(none)]> CREATE USER 'studentuser'@'%' IDENTIFIED BY 'password';
Query OK, 0 rows affected (0.006 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON studentdb.* TO 'studentuser'@'%';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.005 sec)
```

```
MariaDB [(none)]> USE studentdb;
Database changed
MariaDB [studentdb]> CREATE TABLE students (
    ->     student_number VARCHAR(10) PRIMARY KEY,
    ->     first_name VARCHAR(50),
    ->     last_name VARCHAR(50),
    ->     program_enrolled VARCHAR(50),
    ->     graduation_year INT
    -> );
Query OK, 0 rows affected (0.079 sec)

MariaDB [studentdb]> INSERT INTO students (student_number, first_name, last_n
ame, program_enrolled, graduation_year) VALUES
    -> ('110-001', 'Allen', 'Brown', 'Mechanical', 2017),
    -> ('110-002', 'David', 'Brown', 'Mechanical', 2017),
    -> ('110-003', 'Mary', 'Green', 'Mechanical', 2018),
    -> ('110-004', 'Dennis', 'Green', 'Electrical', 2018),
    -> ('110-005', 'Joseph', 'Black', 'Electrical', 2018),
    -> ('110-006', 'Dennis', 'Black', 'Electrical', 2020),
    -> ('110-007', 'Ritchie', 'Salt', 'Computer Science', 2020),
    -> ('110-008', 'Robert', 'Salt', 'Computer Science', 2020),
    -> ('110-009', 'David', 'Suzuki', 'Computer Science', 2020),
    -> ('110-010', 'Mary', 'Chen', 'Computer Science', 2020);
Query OK, 10 rows affected (0.012 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
MariaDB [studentdb]> SELECT * FROM students;
+----------------+------------+-----------+------------------+-----------------+
| student_number | first_name | last_name | program_enrolled | graduation_year |
+----------------+------------+-----------+------------------+-----------------+
| 110-001        | Allen      | Brown     | Mechanical       |            2017 |
| 110-002        | David      | Brown     | Mechanical       |            2017 |
| 110-003        | Mary       | Green     | Mechanical       |            2018 |
| 110-004        | Dennis     | Green     | Electrical       |            2018 |
| 110-005        | Joseph     | Black     | Electrical       |            2018 |
| 110-006        | Dennis     | Black     | Electrical       |            2020 |
| 110-007        | Ritchie    | Salt      | Computer Science |            2020 |
| 110-008        | Robert     | Salt      | Computer Science |            2020 |
| 110-009        | David      | Suzuki    | Computer Science |            2020 |
| 110-010        | Mary       | Chen      | Computer Science |            2020 |
+----------------+------------+-----------+------------------+-----------------+
10 rows in set (0.001 sec)
```

- connect to the database created in step 3 using the new user (with password) Connecting with the new user verifies that the account works, has the correct permissions, and can access and modify the database as intended.

```
[root@system zabbix]# mysql -u studentuser -p -h 127.0.0.1 studentdb
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement
.

MariaDB [studentdb]> SELECT * FROM students;
+----------------+------------+-----------+------------------+-----------------+
| student_number | first_name | last_name | program_enrolled | graduation_year |
+----------------+------------+-----------+------------------+-----------------+
| 110-001        | Allen      | Brown     | Mechanical       |            2017 |
| 110-002        | David      | Brown     | Mechanical       |            2017 |
| 110-003        | Mary       | Green     | Mechanical       |            2018 |
| 110-004        | Dennis     | Green     | Electrical       |            2018 |
| 110-005        | Joseph     | Black     | Electrical       |            2018 |
| 110-006        | Dennis     | Black     | Electrical       |            2020 |
| 110-007        | Ritchie    | Salt      | Computer Science |            2020 |
| 110-008        | Robert     | Salt      | Computer Science |            2020 |
| 110-009        | David      | Suzuki    | Computer Science |            2020 |
| 110-010        | Mary       | Chen      | Computer Science |            2020 |
+----------------+------------+-----------+------------------+-----------------+
10 rows in set (0.001 sec)
```

Implement a web page to show system statistics to the user

- Create directories

```
[root@system ~]# mkdir -p /var/www/html/sysstats/cpu
[root@system ~]# mkdir -p /var/www/html/sysstats/mem
[root@system ~]# mkdir -p /var/www/html/sysstats/disk
[root@system ~]# chown -R apache:apache /var/www/html/sysstats
[root@system ~]# chmod -R 755 /var/www/html/sysstats
```

- The collection script and Make the script executable

```
[root@system ~]# vim collect_stats.sh
```

-b → batch mode (output suitable for scripts)

-n1 → run **once** instead of interactive mode

```
#!/bin/bash
# Collect CPU, memory, disk stats and save to files with timestamp

STATS_DIR="/var/www/html/sysstats"
TIMESTAMP=$(date +"%Y%m%d%H%M")  # sortable timestamp

# CPU usage (user + system)
CPU=$(top -bn2 | grep "Cpu(s)" | tail -n1 | awk '{print $2+$4}')
echo $CPU > $STATS_DIR/cpu/cpu_$TIMESTAMP.txt

# Memory usage
MEM_USED=$(free -m | awk '/Mem:/ {print $3}')
MEM_FREE=$(free -m | awk '/Mem:/ {print $4}')
echo "$MEM_USED $MEM_FREE" > $STATS_DIR/memory/memory_$TIMESTAMP.txt

# Disk usage for root (/)
DISK_USED=$(df -h / | awk 'NR==2 {print $3}')
DISK_FREE=$(df -h / | awk 'NR==2 {print $4}')
echo "$DISK_USED $DISK_FREE" > $STATS_DIR/disk/disk_$TIMESTAMP.txt
```

```
[root@system ~]# chmod +x /root/collect_stats.sh
[root@system ~]# crontab -e
crontab: installing new crontab
Backup of root's previous crontab saved to /root/.cache/crontab/crontab.bak
```

```
# run collect_stats.sh at minute 0 of every hour
#0 * * * * /root/collect_stats.sh

# run every 5 minutes for testing
*/5 * * * * /root/collect_stats.sh
```

- The calculate script and Make the script executable

```
[root@system ~]# vim calculate_avg.sh
[root@system ~]# chmod +x /root/calculate_avg.sh
```

```
#!/bin/bash

# Directories
CPU_DIR="/var/www/html/sysstats/cpu"
MEM_DIR="/var/www/html/sysstats/memory"
DISK_DIR="/var/www/html/sysstats/disk"

# CPU average
awk '{sum+=$1; count++} END {if(count>0) print sum/count}' $CPU_DIR/cpu_*.txt
 > $CPU_DIR/avg_cpu.txt

# Memory average (used and free)
awk '{sum_used+=$1; sum_free+=$2; count++} END {if(count>0) print "Used Avg:"
, sum_used/count, "Free Avg:", sum_free/count}' $MEM_DIR/memory_*.txt > $MEM_
DIR/avg_memory.txt

# Disk average (used and free)
awk '{sum_used+=$1; sum_free+=$2; count++} END {if(count>0) print "Used Avg:"
, sum_used/count, "Free Avg:", sum_free/count}' $DISK_DIR/disk_*.txt > $DISK_
DIR/avg_disk.txt
```

```
# run collect_stats.sh at minute 0 of every hour
#0 * * * * /root/collect_stats.sh

# run every 5 minutes for testing
* * * * * /root/collect_stats.sh

# run ccalculate_avg.sh 5 minutes
5 * * * * /root/calculate_avg.sh

# run every 5 minutes for testing
* * * * * /root/calculate_avg.sh
```

- Install Apache

  Apache is a widely-used web server that serves static and dynamic web content to clients over HTTP/HTTPS. In this task, Apache is required to display system statistics on a web page for users to access.

```
[root@system ~]# yum install httpd -y
Repository AppStream is listed more than once in the configuration
Last metadata expiration check: 1:55:46 ago on Tue 02 Dec 2025 01:24:25 PM EE
T.
Package httpd-2.4.63-1.el10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@system ~]# systemctl start httpd
[root@system ~]# systemctl enable httpd
```

  curl http://localhost/sysstats/

  I access it via `http://localhost:8080/sysstats/index.html`

  because I set up port forwarding in VirtualBox from host port 8080 to the guest's Apache port 80.

  This is when I created separate files for HTML without including them with shell script.

System Statistics

Last updated: 2025-12-02T11:05:02Z (UTC)

CPU Usage — avg: 8.80% (samples: 9)

Memory Usage — avg used: 2903.56 MB, avg free: 341.00 MB (samples: 9)

Disk Usage — avg used: 11755.85 MB (samples: 9)

- Prepare the HTML structure

```
#!/bin/bash
STATS_DIR="/var/www/html/sysstats"
TIMESTAMP=$(date +"%Y%m%d%H%M")

# CPU usage
CPU=$(grep 'cpu ' /proc/stat | awk '{usage=($2+$4)*100/($2+$4+$5); printf "%.
1f", usage}')
echo $CPU > $STATS_DIR/cpu/cpu_$TIMESTAMP.txt

# Memory usage
MEM_USED=$(free -m | awk '/Mem:/ {print $3}')
MEM_FREE=$(free -m | awk '/Mem:/ {print $4}')
echo "$MEM_USED $MEM_FREE" > $STATS_DIR/memory/memory_$TIMESTAMP.txt

# Disk usage
DISK_USED=$(df -m / | awk 'NR==2 {print $3}')
DISK_FREE=$(df -m / | awk 'NR==2 {print $4}')
echo "$DISK_USED $DISK_FREE" > $STATS_DIR/disk/disk_$TIMESTAMP.txt

# --- Optional: regenerate HTML files ---
/root/calculate_avg.sh
```

```
#!/bin/bash

STATS_DIR="/var/www/html/sysstats"
TIMESTAMP=$(date +"%Y-%m-%d %H:%M:%S")

# ---------------- CPU ----------------
CPU_FILES=($STATS_DIR/cpu/cpu_*.txt)
CPU_SUM=0
CPU_COUNT=0
for f in "${CPU_FILES[@]}"; do
    val=$(cat "$f")
    # Make sure it's a number
    if [[ $val =~ ^[0-9.]+$ ]]; then
        CPU_SUM=$(echo "$CPU_SUM + $val" | bc)
        CPU_COUNT=$((CPU_COUNT + 1))
    fi
done
if [ $CPU_COUNT -gt 0 ]; then
    CPU_AVG=$(echo "scale=2; $CPU_SUM / $CPU_COUNT" | bc)
else
    CPU_AVG=0
fi
echo "$CPU_AVG" > $STATS_DIR/cpu/avg_cpu.txt

# ---------------- Memory ----------------
MEM_FILES=($STATS_DIR/memory/memory_*.txt)
MEM_USED_SUM=0
MEM_FREE_SUM=0
MEM_COUNT=0
for f in "${MEM_FILES[@]}"; do
    read used free < "$f"
    # Make sure both are numbers
    if [[ $used =~ ^[0-9]+$ && $free =~ ^[0-9]+$ ]]; then
        MEM_USED_SUM=$((MEM_USED_SUM + used))
                                                        1,1           Top
```

```
        MEM_FREE_SUM=$((MEM_FREE_SUM + free))
        MEM_COUNT=$((MEM_COUNT+1))
    fi
done
if [ $MEM_COUNT -gt 0 ]; then
    MEM_USED_AVG=$(echo "scale=2; $MEM_USED_SUM / $MEM_COUNT" | bc)
    MEM_FREE_AVG=$(echo "scale=2; $MEM_FREE_SUM / $MEM_COUNT" | bc)
else
    MEM_USED_AVG=0
    MEM_FREE_AVG=0
fi
echo "$MEM_USED_AVG $MEM_FREE_AVG" > $STATS_DIR/memory/avg_memory.txt

# ---------------- Disk ----------------
DISK_FILES=($STATS_DIR/disk/disk_*.txt)
DISK_USED_SUM=0
DISK_FREE_SUM=0
DISK_COUNT=0
for f in "${DISK_FILES[@]}"; do
    read used free < "$f"
    if [[ $used =~ ^[0-9]+$ && $free =~ ^[0-9]+$ ]]; then
        DISK_USED_SUM=$((DISK_USED_SUM + used))
        DISK_FREE_SUM=$((DISK_FREE_SUM + free))
        DISK_COUNT=$((DISK_COUNT+1))
    fi
done
if [ $DISK_COUNT -gt 0 ]; then
    DISK_USED_AVG=$(echo "scale=2; $DISK_USED_SUM / $DISK_COUNT" | bc)
    DISK_FREE_AVG=$(echo "scale=2; $DISK_FREE_SUM / $DISK_COUNT" | bc)
else
    DISK_USED_AVG=0
    DISK_FREE_AVG=0
fi
echo "$DISK_USED_AVG $DISK_FREE_AVG" > $STATS_DIR/disk/avg_disk.txt
                                                          40,1          27%
```

```
# ---------------- Generate HTML pages ----------------

# Main index.html
cat > $STATS_DIR/index.html <<EOL
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>System Statistics</title>
<style>
body { font-family: Arial, sans-serif; max-width:900px; margin:2rem auto; pad
ding:0 1rem; }
h1 { color:#222; }
ul { list-style: none; padding:0;}
li { margin: 0.5rem 0; }
a { text-decoration: none; color: #0066cc; }
.meta { color: #666; font-size: 0.9rem;}
</style>
</head>
<body>
<h1>System Statistics</h1>
<p class="meta">Last updated: $TIMESTAMP</p>
<ul>
<li><a href="cpu.html">CPU Usage</a> — avg: $CPU_AVG% (samples: $CPU_COUNT)</
li>
<li><a href="mem.html">Memory Usage</a> — avg used: $MEM_USED_AVG MB, avg fre
e: $MEM_FREE_AVG MB (samples: $MEM_COUNT)</li>
<li><a href="disk.html">Disk Usage</a> — avg used: $DISK_USED_AVG MB, avg fre
e: $DISK_FREE_AVG MB (samples: $DISK_COUNT)</li>
</ul>
</body>
</html>
EOL
```

```
# CPU page
cat > $STATS_DIR/cpu.html <<EOL
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>CPU Usage</title>
</head>
<body>
<h1>CPU Usage</h1>
<p>Average: $CPU_AVG%</p>
<ul>
EOL
for f in "${CPU_FILES[@]}"; do
    ts=$(basename "$f" | sed 's/cpu_//; s/.txt//')
    val=$(cat "$f")
    echo "<li>$ts — $val%</li>" >> $STATS_DIR/cpu.html
done
echo "</ul></body></html>" >> $STATS_DIR/cpu.html

# Memory page
cat > $STATS_DIR/mem.html <<EOL
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Memory Usage</title>
</head>
<body>
<h1>Memory Usage</h1>
<p>Average Used: $MEM_USED_AVG MB, Average Free: $MEM_FREE_AVG MB</p>
<ul>
```

```
for f in "${MEM_FILES[@]}"; do
    ts=$(basename "$f" | sed 's/memory_//; s/.txt//')
    read used free < "$f"
    echo "<li>$ts — Used: $used MB, Free: $free MB</li>" >> $STATS_DIR/mem.ht
ml
done
echo "</ul></body></html>" >> $STATS_DIR/mem.html

# Disk page
cat > $STATS_DIR/disk.html <<EOL
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Disk Usage</title>
</head>
<body>
<h1>Disk Usage</h1>
<p>Average Used: $DISK_USED_AVG MB, Average Free: $DISK_FREE_AVG MB</p>
<ul>
EOL
for f in "${DISK_FILES[@]}"; do
    ts=$(basename "$f" | sed 's/disk_//; s/.txt//')
    read used free < "$f"
    echo "<li>$ts — Used: $used MB, Free: $free MB</li>" >> $STATS_DIR/disk.h
tml
done
echo "</ul></body></html>" >> $STATS_DIR/disk.html
```

System Statistics

Last updated: 2025-12-02 16:20:02

CPU Usage — avg: 8.73% (samples: 153)

Memory Usage — avg used: 1432.00 MB, avg free: 1771.63 MB (samples: 41)

Disk Usage — avg used: 11807.00 MB, avg free: 13730.00 MB (samples: 5)

CPU Usage

Average: 8.73%

- 202511280150 — 15.1%
- 202511280151 — 20%
- 202511280152 — 0%
- 202511280153 — 0%
- 202511280154 — 14.8%
- 202511280155 — 4.5%
- 202511280156 — 0%
- 202511280157 — 0%
- 202511280158 — 0%
- 202511280159 — 0%
- 202511280200 — 8.6%

Disk Usage

Average Used: 11806.00 MB, Average Free: 13731.00 MB

- 202512021719 — Used: 11806 MB, Free: 13731 MB
- 202512021720 — Used: 11806 MB, Free: 13731 MB
- 202512021725 — Used: 11806 MB, Free: 13731 MB